手写小GPT学习笔记 | Day16 词嵌入层+位置编码

Day16核心任务：补齐GPT模型的**输入层模块**，实现**词嵌入（Token Embedding）**和**位置编码（Positional Encoding）**，解决Transformer“无时序感知能力”的痛点，把文本token序列转为模型可处理的带位置信息的特征，完美衔接Day15的Block堆叠骨架。

一、Day16 学习目标

- **理解嵌入逻辑**：搞懂词嵌入的映射原理、维度匹配规则，明白Token到向量的转换过程
- **掌握位置编码**：吃透sin/cos固定位置编码、可学习位置编码两种实现方式，区分适用场景
- **完成模块拼接**：把嵌入层+位置编码+GPT堆叠模块串联，实现文本token→模型输入的完整流程
- **验证维度合规**：确保嵌入、位置编码、堆叠模块全程维度守恒，无广播报错
- **夯实前置依赖**：为Day17完整GPT模型定义做好铺垫，预留接口适配输出层

二、前置核心原理

🔑 **核心痛点**：Transformer是并行运算结构，不像RNN有天然时序感，必须**手动注入位置信息**，否则模型无法区分词序。

1. 词嵌入（Token Embedding）

- **作用**：将离散的文本token（数字索引，如0=pad、1=你、2=好）映射为连续稠密向量，让模型能计算语义相似度
- **维度规则**：输入token序列shape `[batch, seq_len]` → 嵌入后shape `[batch, seq_len, d_model]`，必须和Block的d_model完全一致
- **实现方式**：PyTorch内置 `nn.Embedding`，参数为词表大小(vocab_size)、模型维度(d_model)

2. 位置编码（Positional Encoding）

位置编码是给每个位置分配专属向量，和词嵌入逐元素相加，注入时序信息，主流分两种：

- **固定位置编码（sin/cos）**：无需训练，靠三角函数生成，适合小模型、Demo，泛化性强

- **可学习位置编码（Learnable PE）**：随机初始化后随模型训练优化，效果更优，GPT系列默认采用

关键规则：位置编码维度必须和词嵌入完全一致（`[seq_len, d_model]`），相加时自动广播适配batch维度

三、核心代码实现

复用Day15的Transformer Block堆叠代码，新增嵌入层、两种位置编码模块，最终封装**GPT输入编码器**，代码可直接运行、注释详尽。

Day16 核心代码：词嵌入+位置编码+模块串联

```python
import torch
import torch.nn as nn
import torch.nn.functional as F
import math
import os

os.environ["PYTORCH_ENABLE_MPS_FALLBACK"] = "1"

# ===================== 复用Day14、Day15基础组件（无修改）=====================
# 掩码工具、注意力、FFN、TransformerBlock、GPTStack 直接沿用
# 此处省略重复代码，直接调用前期已实现模块
# ------------------- 缩放点积注意力 -------------------
class ScaledDotProductAttention(nn.Module):
    def __init__(self, dropout=0.1):
        super().__init__()
        self.dropout = nn.Dropout(dropout)
    def forward(self, q, k, v, mask=None):
        d_k = q.size(-1)
        scores = torch.matmul(q, k.transpose(-2, -1)) / torch.sqrt(torch.tensor(d_k, dtype=torch.float32))
        if mask is not None:
            scores = scores.masked_fill(mask == True, -1e9)
        attn = F.softmax(scores, dim=-1)
        attn = self.dropout(attn)
        return torch.matmul(attn, v), attn

# ------------------- 多头注意力 -------------------
class MultiHeadAttention(nn.Module):
    def __init__(self, d_model, num_heads, dropout=0.1):
        super().__init__()
        assert d_model % num_heads == 0
        self.d_model = d_model
        self.num_heads = num_heads
        self.d_k = d_model // num_heads
```

```python
        self.w_q = nn.Linear(d_model, d_model)
        self.w_k = nn.Linear(d_model, d_model)
        self.w_v = nn.Linear(d_model, d_model)
        self.w_o = nn.Linear(d_model, d_model)
        self.attn = ScaledDotProductAttention(dropout)
        self.dropout = nn.Dropout(dropout)

    def split_heads(self, x, batch_size):
        return x.view(batch_size, -1, self.num_heads, self.d_k).transpose(1, 2)

    def combine_heads(self, x, batch_size):
        return x.transpose(1, 2).contiguous().view(batch_size, -1, self.d_model)

    def forward(self, q, k, v, mask=None):
        batch_size = q.size(0)
        q_proj = self.split_heads(self.w_q(q), batch_size)
        k_proj = self.split_heads(self.w_k(k), batch_size)
        v_proj = self.split_heads(self.w_v(v), batch_size)
        attn_out, _ = self.attn(q_proj, k_proj, v_proj, mask)
        out = self.combine_heads(attn_out, batch_size)
        return self.dropout(self.w_o(out))

# ------------------- FFN & Transformer Block -------------------
class FeedForwardNetwork(nn.Module):
    def __init__(self, d_model, dropout=0.1, expansion=4):
        super().__init__()
        self.linear1 = nn.Linear(d_model, d_model*expansion)
        self.linear2 = nn.Linear(d_model*expansion, d_model)
        self.gelu = nn.GELU()
        self.dropout = nn.Dropout(dropout)
    def forward(self, x):
        return self.linear2(self.dropout(self.gelu(self.linear1(x))))

class TransformerBlock(nn.Module):
    def __init__(self, d_model, num_heads, dropout=0.1):
        super().__init__()
        self.attn = MultiHeadAttention(d_model, num_heads, dropout)
        self.ffn = FeedForwardNetwork(d_model, dropout)
        self.norm1 = nn.LayerNorm(d_model)
        self.norm2 = nn.LayerNorm(d_model)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x, mask=None):
        residual = x
        x = self.norm1(x)
        x = residual + self.dropout(self.attn(x, x, x, mask))
```

```
        residual = x
        x = self.norm2(x)
        x = residual + self.dropout(self.ffn(x))
        return x

# ------------------- Block堆叠模块 -------------------
class GPTStack(nn.Module):
```

四、关键细节精讲

1. 嵌入缩放（乘√d_model）

词嵌入初始化后数值较小，直接加位置编码会导致位置信息占比过低，乘以 `sqrt(d_model)` 能平衡两者权重，是GPT系列的标准操作，避免模型忽视位置信息。

2. 两种位置编码对比

类型	优点	缺点	适用场景
sin/cos固定编码	无需训练、推理快、泛化强、不占参数量	效果略差，长序列适配性一般	Demo测试、小模型、资源受限场景
可学习编码	效果优、贴合数据、长序列表现好	增加参数量、需训练优化	正式训练、GPT原生架构、追求效果

3. 缓冲区与参数区别

- **register_buffer**：固定位置编码用，不参与梯度更新，随模型保存加载
- **nn.Parameter**：可学习编码用，参与训练、更新权重

五、Day16 避坑清单

💡 **高频报错&避坑点**

- **维度不匹配**：词嵌入、位置编码、Block的d_model必须完全一致，否则无法相加
- **位置编码广播失败**：固定编码shape是[max_len, d_model]，需截取seq_len长度再用
- **缺失嵌入缩放**：位置信息被稀释，模型学不到词序，生成文本逻辑混乱
- **max_len设置过小**：超出长度后无位置编码，报错或特征异常

- **token从0开始**：pad_idx=0需保留，词表索引从0开始，避免嵌入层越界

六、今日成果验收+明日预习

今日达标标准

- ✅ 运行测试脚本无报错，token→嵌入→堆叠模块全程维度守恒
- ✅ 理解两种位置编码原理，能自由切换使用
- ✅ 掌握嵌入缩放、缓冲区注册等细节技巧
- ✅ 完成输入层与解码器骨架的无缝拼接

明日（Day17）预习

整合**嵌入层+Block堆叠+输出线性层+Softmax**，定义**完整GPT模型**，实现模型封装，为后续推理、训练做准备。

🎯 Day16任务完成！GPT的输入模块已彻底补齐，距离完整可运行的小GPT只剩最后一步封装，继续加油～

（注：文档部分内容可能由 AI 生成）